SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192S24438RC

QUESTIONS 10 / 10 completed

Binary Tree-Inorde

BINARY TREE-INORDER REVERSAL

Write a C program to create a Binary Tree and perform Recursive Inorder Traversal. Consider an organizational hierarchy management system where employee records are stored in a binary tree structure. The program should create the tree dynamically and display employee IDs in sorted order using inorder traversal to facilitate systematic record verification. Sample Input: Enter Nodes: 50 30 70 20 40 60 80. Sample Output: Inorder

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);
```

OUTPUT

```
Inorder Traversal: 20 30 40 50 60 70 80
```

INPUT

```
50 30 70 20 40 60 80
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BT- Rec. Postorder

BT- REC. POSTORDER TRAVERSE

Write a C program to create a Binary Tree and perform Recursive Postorder Traversal. Consider a software uninstallation utility where dependent components must be removed before the parent application. The program should display the deletion order using postorder traversal. Sample Input: Enter Nodes: 25 15 35 10 20 30 40. Sample Output: Postorder Traversal: 10 20 15 30 40 35 25.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);
```

OUTPUT

```
Postorder Traversal: 10 20 15 30 40 35 25
```

INPUT

```
25 15 35 10 20 30 40
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BST- In,Pre,Post Tr:

BST-IN,PRE,POST TRAVERSE

Write a C program to create a Binary Search Tree (BST) and perform Inorder, Preorder, and Postorder Traversals. Consider a student record management system where roll numbers are stored in a BST and different traversals are used for various reporting purposes. Sample Input: Enter Nodes: 45 25 65 15 35 55 75. Sample Output: Inorder: 15 25 35 45 55 65 75. Preorder: 45 25 15 35 65 55 75. Postorder: 15 35 25 55 75 65 45.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);
```

OUTPUT

```
Inorder: 15 25 35 45 55 65 75. Preorder: 45 25 15 35 65 55 75. Postorder: 15 35 25 55 75 65 45.
```

INPUT

```
45 25 65 15 35 55 75
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BST- Finding Heigl

BST- FINDING HEIGHT

Write a C program to find the height of a Binary Tree. Consider a network routing structure where devices are arranged hierarchically and the tree height represents the maximum communication depth. The program should determine the height of the constructed tree. Sample Input: Nodes: 1 2 3 4 5 6 7. Sample Output: Height of Binary Tree: 3.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insertLevelOrder(int arr[], int i, int n) {
    struct Node* root = NULL;
    if (i < n) {
        root = createNode(arr[i]);
        root->left = insertLevelOrder(arr, 2 * i + 1, n);
        root->right = insertLevelOrder(arr, 2 * i + 2, n);
```

INPUT

1 2 3 4 5 6 7

OUTPUT

```
Height of Binary Tree: 3
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BST- Count Nodes

BST- COUNT NODES

Write a C program to count the total number of nodes, leaf nodes, and non-leaf nodes in a Binary Tree. Consider a company's organizational structure where leaf nodes represent employees without subordinates and non-leaf nodes represent managers. Sample Input: Nodes: 50 30 70 20 40 60 80. Sample Output: Total Nodes: 7. Leaf Nodes: 4. Non-Leaf Nodes: 3.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
```

OUTPUT

```
Total Nodes: 7. Leaf Nodes: 4. Non-Leaf Nodes: 3.
```

INPUT

```
50 30 70 20 40 60 80
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BST- Min, Max Vali

BST- MIN, MAX VALUES

Write a C program to find the minimum and maximum values in a Binary Search Tree. Consider a sales database where transaction amounts are stored in a BST and management wants to identify the smallest and largest transactions. Sample Input: Nodes: 100 50 150 25 75 125 175. Sample Output: Minimum Value: 25. Maximum Value: 175.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
```

INPUT

```
100 50 150 25 75 125
175
```

OUTPUT

```
Minimum Value: 25. Maximum Value: 175.
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BST- Deletion

BST- DELETION

Write a C program to delete a node from a Binary Search Tree and display the tree using inorder traversal after deletion. Consider a library management system where discontinued book IDs must be removed from the catalog. Sample Input: Nodes: 50 30 70 20 40 60 80. Delete Node: 30. Sample Output: Inorder Traversal after Deletion: 20 40 50 60 70 80.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
```

INPUT

30

OUTPUT

```
Inorder Traversal after Deletion: 20 40 50 60 70 80
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

BST- Mirroring

BST- MIRRORING

Write a C program to mirror a Binary Tree and display the inorder traversal before and after mirroring. Consider a graphical application where tree structures need to be reflected horizontally for visualization purposes. Sample Input: Nodes: 10 5 15 3 7 12 18. Sample Output: Original Inorder: 3 5 7 10 12 15 18. Mirrored Inorder: 18 15 12 10 7 5 3.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insert(struct Node* root, int data) {
    if (root == NULL) return createNode(data);
    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
```

OUTPUT

```
Original Inorder: 3 5 7 10 12 15 18. Mirrored Inorder: 18 15 12 10 7 5 3.
```

INPUT

```
10 5 15 3 7 12 18
```